

Computer Architectures 02LSEOV	Delivery date: By 2:00 AM on October, 23 2024
Laboratory 3	Expected delivery of lab_03.zip must include: - program_1_a.s, program_1_b.s, and program_1_c.s - This file, filled with information and possibly compiled in a pdf format.

This lab will explore some of the concepts seen during the lessons, such as hazards, rescheduling, and loop unrolling. The first thing to do is to configure the WinMIPS64 simulator with the *Initial Configuration* provided below:

- Integer ALU: 1 clock cycle
- Data memory: 1 clock cycle
- Code address bus: 12
- Data address bus: 12
- FP arithmetic unit: pipelined, 4 clock cycles
- FP multiplier unit: pipelined, 6 clock cycles
- FP divider unit: not pipelined, 30 clock cycles
- Forwarding is enabled
- Branch prediction is disabled
- Branch delay slot is disabled

1) Enhance the assembly program you created in the previous lab called **program_1.s**:

```
int m=1 /* 64 bit */
double a, b
for (i = 31; i >= 0; i--){
    if (i is a multiple of 3) {
        a = v1[i] / ((double) m<< i) /*logic shift */
        m = (int) a
    } else {
        a = v1[i] * ((double) m* i)
        m = (int) a
    }
    v4[i] = a*v1[i] - v2[i];
    v5[i] = v4[i]/v3[i] - b;
    v6[i] = (v4[i]-v1[i])*v5[i];
}
```

- Manually detect the different data, structural, and control hazards that cause a pipeline stall.
- Optimize the program by re-scheduling the program instructions to eliminate as many hazards as possible. Manually calculate the number of clock cycles for the new program (**program_1_a.s**) to execute and compare the results with those obtained by the simulator.
- Starting from **program_1_a.s**, enable the *branch delay slot* and re-schedule some instructions to improve the previous program execution time. Manually

calculate the number of clock cycles needed by the new program (**program_1_b.s**) to execute and compare the results obtained with those obtained by the simulator.

- d. Unroll the program (**program_1_b.s**) 3 times; if necessary, re-schedule some instructions and increase the number of registers used. Manually calculate the number of clock cycles to execute the new program (**program_1_c.s**) and compare the results obtained with those obtained by the simulator.

Complete the following table with the obtained results:

Program	program_1.s	program_1_a.s	program_1_b.s	program_1_c.s
Clock cycle computation				
By hand	2680	2680	2680	2626
By simulation	3844	3663	3474	3131

- 2) Collect the Cycles Per Instruction (CPI) from the simulator for different programs

	program_1.s	program_1_a.s	program_1_b.s	program_1_c.s
CPI	<u>5.339</u>	<u>5.088</u>	<u>4.62</u>	<u>4.701</u>

Compare the results obtained in 1) and provide some explanation if the results are different.

Eventual explanation: The improvement in the number of cycles from program to program was achieved through various optimizations. In the original program, each iteration of the loop performed conditional checks and jumps that caused stalls in the pipeline. With the transition to 'program_1_a.s', instruction rescheduling reduced stalls due to data and resource dependencies, improving flow and decreasing the required cycles. In 'program_1_b.s', by enabling the branch delay slot, we were able to insert useful instructions into the cycles lost during conditional jumps, enhancing efficiency. Finally, in 'program_1_c.s', loop unrolling allowed us to completely eliminate the conditional checks and jumps for each iteration, executing multiple operations per cycle and significantly reducing the overall number of cycles needed. These optimizations gradually improved the program's efficiency.